

Commerce Data Standard (CDS)

Shopify Store Setup Runbook (CDS v0.2)

Field	Value
Status	v0.2 Companion — Informative (profile companion; not normative standard text)
Release	CDS v0.2
Date	2026-08-17
Doc type	How-to runbook (operator-facing)
Applies to	New Shopify store, apparel + homewares retailer, small team, PIM-first operation
Profile stack	CDS Core → <code>cds.profile.apparel.v0_2</code> + <code>cds.profile.homewares.v0_2</code> (CDS-1500) → CDS-SHOPIFY-0.2 (CDS-900 §4) → this store's organisation profile
Normative status	Entirely informative . Every step cites the requirement IDs it implements; the cited chapters govern. Where this runbook and a CDS v0.2 chapter disagree, the chapter wins.
Supersedes (operationally)	Legacy implementation runbook (its sequencing doctrine is retained; its v0.1-era details are re-based on CDS v0.2)
Depends on	CDS-200, CDS-300, CDS-400, CDS-500, CDS-600, CDS-900, CDS-1300, CDS-1500; starter data: <code>CDS_Reference_Dictionary/</code> (Appendix E contract)

TL;DR — the whole runbook in one page

What you're building. A Shopify store where the PIM is the single source of truth. Product data is described once in the PIM, projected out to Shopify and Google, then *read back and compared* — so every field on every product shows Green (matches), Amber (missing/pending/accepted difference) or Red (contradiction), with a machine-readable reason underneath. Shopify becomes a screen you publish to, not a place you edit.

The five ideas everything else hangs off:

- 1. Design before platform, platform before products.** Taxonomy and dictionaries on paper first; Shopify structures second; products last. Retrofitting order is the expensive mistake (Phases 0–3 before 4–8 before 9).
- 2. Controlled words, not free text.** Colours, materials, fits etc. come from the seed dictionaries — a supplier's "French Navy" maps to canonical `french_navy`, displays as "French Navy", filters as "Blue". Unknown values queue for a human; nothing invents vocabulary (Phase 2).
- 3. Everything customers filter by is a governed metafield; tags only drive collections.** Tags are generated from the category tree, never hand-typed, never exposed as filters (Phases 3–6).
- 4. One writer per field.** The PIM owns product content; Shopify staff edits to PIM-owned fields become detected *drift*, not silent truth. Google gets the landing-page colour name, never the broad family (Phase 8).

5. **Published ≠ correct.** A successful API call proves nothing; the read-back loop in Phase 10 is what proves the store actually says what the PIM says.

You will be stopped for six OWNER DECISIONS: small-team profile (0), category tie-break order (1), colour family set (2), tag suffix format (3), metafield posture — visible vs app-owned (4, recommendation: visible + drift detection), and the go/no-go gates between product waves (9).

Done looks like: the completion checklist passes — PIM authoritative, expected state reproducible, read-back stored separately, every difference classified with a reason, and the storefront filter shows "Blue" while the product page shows "French Navy". That is also, not coincidentally, the CDS vertical-slice acceptance test.

Time-boxing honestly: Phases 0–3 are thinking work (days, mostly yours); Phases 4–8 are configuration (days); Phase 9 scales with catalogue size; Phase 10 runs forever — it's the point.

How to use this document. Work top to bottom. Each step states *what to do, why* (one line), and the CDS requirement IDs it implements — read the cited rules before improvising. Steps marked **OWNER DECISION** stop the line until the owner decides and the decision is recorded in the organisation profile. The sequencing doctrine is the legacy store's proven order — design before platform, platform structures before products — updated so that the PIM schema and dictionaries come first, because in vo.2 the PIM is authoritative for everything Shopify displays.

The sequence at a glance:

```
Phase 0  Prerequisites and declarations
Phase 1  Taxonomy design (on paper, with tie-break order + decision log)
Phase 2  Dictionaries and attribute schema in the PIM (seed from the CDS Reference Dictionary
package)
Phase 3  Tag vocabulary and collection projection design
Phase 4  Shopify store bootstrap (permissions, app, metafield posture + definitions)
Phase 5  Automated collections
Phase 6  Search & Discovery filters (within verified limits)
Phase 7  Navigation menu
Phase 8  PIM channel wiring (channel profile, mappings, ownership, GMC)
Phase 9  Products – pilot cohort, then waves
Phase 10 Switch on publish → observe → verify (+ two queued empirical checks)
        Completion checklist (vertical-slice acceptance)
```

Build in this order and adding the 500th product is as fast as adding the first. Build in any other order and you will retrofit for months — the legacy runbook's core lesson, unchanged.

Phase 0 — Prerequisites and Declarations

0.1 Confirm the Shopify store and plan

Create the store (or claim the dev store) on a current standard plan. No feature in this runbook requires Shopify Plus. If a variant decomposition strategy involving **combined listings** is ever contemplated,

confirm plan availability first — it has historically been Plus-restricted and is not verified for the current plan. **Why:** the whole runbook must run within the plan's declared capabilities. Implements: CDS900-R002, CDS900-R023 (preflight against declared limits); caveat per CDS-900 §4.5.1.

0.2 Confirm PIM readiness

Before anything is configured in Shopify, the PIM must be able to:

1. Store canonical values separately from display labels, facet values and channel representations (value layers).
2. Persist an **expected channel state** per product/field that is reproducible from versioned inputs (canonical revision + mapping version + dictionary version).
3. Store read-back (observed) values **separately** from canonical values — never overwriting them.
4. Retain observation records **append-only** per field and channel (previous observed value, new observed value, detection interval must be recoverable).
5. Name visible fields per the CDS-300 registry (STD_, CAT_, VAR_, MF_, CH_, OBS_, QA_, ...).

If any of these is missing, stop: the verification loop in Phase 10 cannot be honest without them. **Why:** the PIM is the master layer for the whole lifecycle; these five capabilities are the load-bearing floor. Implements: CDS500-R001, CDS500-R005, CDS500-R022, CDS500-R077a, CDS200-R031, CDS300-R009.

0.3 Confirm the dictionary seed package is present

Locate `CDS_Reference_Dictionary/` and verify it against its `manifest.json` and `SHA256SUMS.txt` (the manifest is the version authority — the folder is deliberately unversioned; every CSV row carries the Appendix E column contract: `dictionary_key`, `row_type`, `value_id`, `label`, `aliases`, `facet_ids`, `status`, `provenance`, `version`, `locale`). **Why:** Phase 2 seeds governed dictionaries from this package; a tampered or contract-violating package poisons everything downstream. Implements: CDS-1500 Appendix E (column contract), CDS1500-R011.

0.4 OWNER DECISION — declare the small-team profile

A small team may declare the **small-team profile** in its conformance claim: combined migration credentials are permitted (with attributable audit logging reviewed at wave exit) and the intervention-free readiness demonstration is waived where operators and project team are the same people. All other requirements apply unchanged; functions are hats, not headcount — every function still gets a **named owner**. **Standing recommendation:** declare it. This store is exactly the profile's target. **Why:** it makes the conformance claim honest for a small team without waiving any evidence. Implements: CDS1300-R003, CDS1300-R004, CDS1300-R005 (Table 2-1), CDS1300-R053.

0.5 Open the organisation profile document

Open `CDS_Adoption_Workbook_v0.2.md` and use it as the organisation profile, or represent the same fields in the PIM. It accumulates every decision this runbook produces: tie-break order, tag format, metafield posture, facet sets, channel capability declarations, named function owners. Every divergence from the

platform/industry profiles gets documented here. **Why:** an organisation profile may extend the stack but must document every divergence. Implements: CDS900-R005, CDS900-R006.

Phase 1 — Taxonomy Design (on paper, before any system)

This is the legacy runbook's most important phase, carried forward intact: the taxonomy is a document — spreadsheet or whiteboard — settled before Shopify or the PIM is touched.

1.1 Design the internal category tree

Draft the internal tree for the full intended range (apparel + homewares), 2–4 levels deep (3 is the sweet spot for 100–1,000 products). Recommended shape: **Product Family** → **Product Type** → **Category** (e.g. Apparel > Women > Tops > Shirts; Homewares > Living Room > Tables > Side Tables). For every node apply the scale test: will it hold ≥ 5 products; would a customer seek it specifically; can you write one sentence saying what belongs and what does not. Merge failures into the parent. **Why:** every collection, filter, menu item and channel mapping downstream is a projection of this tree. Implements: CDS200-R013, CDS200-R014, CDS200-R016.

1.2 Enforce one primary category per product type

Every product will get **exactly one primary internal category** — the most specific applicable node. Products may later appear in many collections and carry secondary placements, but classification is single-homed. Distinguish *taxonomy overlap* (categories not distinct — fix the boundaries now) from *merchandising overlap* (one function, two audiences — solved later with `merch_*` tags and cross-links, never a second category). **Why:** single-homing is what makes collections, breadcrumbs, inheritance and channel mappings deterministic. Implements: CDS200-R015, CDS200-R016, CDS200-R019, CDS200-R022, CDS600-R012, CDS600-R091.

1.3 OWNER DECISION — declare the tie-break order

When a product plausibly fits two categories, resolution uses an **organisation-declared, documented, ordered rule set**, applied in order until one rule decides, and applied consistently forever after. Record the declared order in the organisation profile. **Recommended default (adopt unless you have a reason not to):** 1. **Customer mental model** — where would the buying customer navigate or search? 2. **Dominant use** — what is the product primarily designed to do? 3. **Material / form** — what is it physically, where use is ambiguous? 4. **Commercial priority** — where do its assortment peers sit?

(The legacy runbook's four rules — primary use, purchase intent, brand classification, majority audience — are superseded by this list; "purchase intent" survives as rule 1, "majority audience" as rule 4. The owner has flagged this order may be revisited after practical use.) **Why:** the rule matters less than declaring one and never re-litigating it per product. Implements: CDS200-R017.

1.4 Start the taxonomy decision log — and keep it forever

Create the decision log now, as part of the taxonomy document, and migrate it into the PIM when categories are entered. Every tie-break entry records at minimum: **product type/product | category chosen | rule applied | date**. Subsequent matching products follow the logged entry; changing a decision requires a superseding entry, never an ad-hoc different assignment. Example entries for this range:

Product type	Category chosen	Rule applied	Date
Unisex hoodies	Apparel > Men > Tops > Hoodies (+ merch_unisex tag for Women browse)	4 commercial priority	2026-08-04
Sports bras	Apparel > Women > Activewear	1 customer mental model	2026-08-04
Outdoor cushions	Homewares > Outdoor > Cushions	1 customer mental model	2026-08-04
Table lamps	Homewares > Lighting > Table Lamps	2 dominant use	2026-08-04

Also decide and log the **accessories rule** once (accessories under their parent product category, or in their own node) and apply it everywhere. **Why:** the log is the mechanism that makes the tie-break order enforceable by future editors. Implements: CDS200-R018.

1.5 Map each category to Shopify's Standard Product Taxonomy

For every **final** internal category, record one Shopify Standard Product Taxonomy category (browse at shopify.github.io/product-taxonomy). Store the mapping **on the category entity** in the taxonomy document (later the PIM), inherited by products; product-level exceptions only as documented, governed overrides. Assigning this category on a Shopify product is what **unlocks the pre-built category metafields** (apparel: size, colour, fabric, neckline, sleeve length...; homewares vary by node) — the raw material for Phase 6 filters.

```
Internal: Apparel > Women > Tops > Shirts
Shopify: Apparel & Accessories > Clothing > Clothing Tops > Shirts
Internal: Homewares > Living Room > Tables > Side Tables
Shopify: Furniture > Tables > Accent Tables (closest current node)
```

Why: one mapping per category gives every product the right `product_category` and unlocked attribute fields with zero per-product work. Implements: CDS900-R014, CDS900-R015, CDS900-R016, CDS200-R020; platform facts per CDS-900 §4.3 [S1].

1.6 Map each category to the Google product taxonomy

On the same category records, store the Google taxonomy mapping as a **channel mapping** (`CH_google_product_category` — a taxonomy ID *or* full breadcrumb, never both). Map at the highest level where Google's match is right and let children inherit; override a child only where Google has a distinctly better node. `CH_google_product_type` will later be derived from the internal category path (broad → specific) — do not hand-author it. **Why:** Google classification is a per-category mapping, set once and inherited — never a per-product chore and never canonical truth. Implements: CDS900-R042, CDS900-R043, CDS900-R044, CDS200-R020, CDS200-R005.

1.7 Define the Shopify product type vocabulary

List one controlled, single-concept product type per final category (Shirt, Side Table, Decorative Cushion). It is a merchant vocabulary distinct from both the category tree and the Shopify taxonomy — never a concatenation of attributes ("Blue Linen Relaxed Shirt" is wrong). **Why:** product type is a third classification surface; keeping it controlled keeps it useful. Implements: CDS900-R014, CDS900-R017.

Phase 2 — Dictionaries and Attribute Schema in the PIM

Dictionaries before bulk data, always. This phase happens in the PIM (or its governing documents) — still nothing in Shopify.

2.1 Seed governed dictionaries from the CDS Reference Dictionary package

Import the package CSVs from `CDS_Reference_Dictionary/` as the initial content of this organisation's **governed** dictionaries: colour facets, colour reference values, `material_apparel`, `material_homewares`, `material_facet`, patterns, fits, styles, occasions, rooms, finishes, shapes. On adoption they become governed data under your control: stable `value_ids`, lifecycle status, provenance, aliases preserved; a published canonical code is never silently redefined — changes go through deprecation with replacement mappings. **Why:** requirement levels bind to governed dictionaries, not to loose CSVs; adopting the seed verbatim is the fast, conformant start. Implements: CDS1500-R004, CDS1500-R011, CDS1500-R050.

2.2 OWNER DECISION — colour facet family set

The colour facet baseline (CDS-1500 Appendix C) ships 18 families. **Recommended: run the ~12-family core** — black, white, grey, beige, brown, red, orange, yellow, green, blue, purple, pink, plus **multicolour** as a status value — and enable extensions only where the assortment warrants them: **gold/silver** (metallic decor, jewellery), **clear** (glassware), **cream/natural** (textiles, natural-fibre homewares). For this apparel + homewares range, core-12 **plus natural and clear** is a sensible opening declaration; extensions later follow normal facet governance. Record the declared set in the organisation profile. Note the pre-split ambiguous shades are already handled in the seed:

`charcoal_grey/charcoal_black`, `rust_orange/rust_brown`, `tan_beige/tan_brown` — never re-merge them. **Why:** a shorter family list filters better; the split shades keep facet membership deterministic at dictionary level. Implements: CDS1500-R017, CDS1500-R020, CDS1500-R048, CDS1500-R049; baseline per CDS-1500 Appendix C.1–C.3.

2.3 Confirm the shared colour architecture

One colour-family facet vocabulary across apparel **and** homewares (customers see the same "Blue" everywhere); canonical shade dictionaries beneath may differ per range. Multi-facet shades (teal → green|blue) keep their ordered facet list, primary first. Product-level facet overrides exist only as governed override records with provenance — never dictionary edits per product. **Why:** consistent filter language store-wide; deterministic facet membership. Implements: CDS1500-R044, CDS1500-R019, CDS1500-R021, CDS600-R034.

2.4 Create Attribute Definitions from the industry baselines

In the PIM, create an Attribute Definition for every attribute in CDS-1500 Appendix A (apparel) and Appendix B (homewares), with the baseline requirement levels (**R** required / **C** conditional / **REC** recommended / **O** optional) evaluated per product family — never as one flat universal list. Non-negotiables to get right now:

Attribute	Key rule
VAR_colour	Variant scope where colour distinguishes sellable units; stores the canonical value_id (colour_french_navy), never text
MF_colour_display / MF_colour_facet	Projections derived from the variants' canonical values — never independently authored
VAR_size_label + VAR_size_system + VAR_size_sort_key	Size always carries its system (AU_WOMENS_NUMERIC, ALPHA, ...) and a semantic sort key
MF_fibre_composition	Numeric percentages against canonical material ids, totalling 100% per component; fabric type is a separate field, never a substitute
MF_material_primary / MF_material_facet	Facet family never erases the specific canonical material
Dimensions (MF_height_mm etc.), capacity, weight	Typed numeric values with declared units — display strings alone are non-conformant
Claims (MF_organic_claim, MF_recycled_content_percent)	Stored separately from material identity; require evidence

Each definition also declares: scope, data type, dictionary binding, **channel source layer** (which value layer — canonical, display or facet — feeds each channel), publication mapping, and **comparison strategy** (from the normalisation registry) — these feed Phases 8 and 10 directly. **Why:** the Attribute Definition is where behaviour lives; products only store values. Implements: CDS1500-R001, CDS1500-R002, CDS1500-R006, CDS1500-R009, CDS1500-R012–R014, CDS1500-R023–R028, CDS1500-R034, CDS1500-R037, CDS200-R023–R026, CDS500-R055.

2.5 Plan the known-missing dictionaries

The seed package (0.8.0) ships size systems (Chapter 5, including apparel, footwear, bedding, ring, bike-frame and head-circumference systems), fabric types (Chapter 3) and descriptive care instructions (Chapter 4). It does not ship: the trademarked care-symbol set, country of origin (generate from ISO 3166 at adoption), component roles, season/weather relevance, use environment. For launch you need **size systems** governed at minimum (apparel is unpublishable without them): adopt the Chapter 5 values as your governed size-system dictionary now and add any brand-specific systems (e.g. a numeric womenswear system with its sort keys); queue the rest. Until governed, these fields cannot satisfy R-level requirements. If the store sells into more than one country, also open the jurisdiction requirement register (CDS-1500 §2.2) from the Appendix G seed for apparel and homewares. **Why:** disclosure of the gap is in the package; closing the size-system gap is a launch blocker for apparel. Implements: CDS-1500 Appendix E.3, CDS1500-R011, CDS1500-R026, CDS1500-R033.

2.6 Register the namespace registry entries

Confirm the PIM's namespace registry covers every prefix this store will use — at minimum `STD_`, `CAT_`, `VAR_`, `MF_`, `CH_`, `SEO_`, `MED_`, `WF_`, `QA_`, `OBS_`, `IMPORT_`, `AI_` — each with meaning, owner, scope, publication behaviour and lifecycle status. `WF_`, `QA_`, `OBS_`, `IMPORT_`, `AI_` are never published to customer-facing channels. New channel fields use `CH_` (`DF_` is deprecated; migration aliases only). **Why:** CDS-300 is the registry of record; every visible identifier must resolve against it. Implements: CDS300-R009, CDS300-R010, CDS300-R012, CDS900-R069.

Phase 3 — Tag Vocabulary and Collection Projection Design

Tags in v0.2 are **governed integration signals** — projections regenerated from rules, never the store of structured facts. Design the whole vocabulary before Shopify sees a single tag.

3.1 Derive `collection_*` tags from the taxonomy — never invent them

Every browsable taxonomy node becomes exactly one `collection_` tag, computed by the PIM from the category path at publication. A product in `Apparel > Women > Tops > Shirts` carries `collection_Women`, `collection_Tops`, `collection_Shirts` — generated, not typed. Nobody ever types a `collection_*` tag by hand. **Why:** the tag layer is a projection of classification; deriving it makes collections self-maintaining. Implements: CDS200-R007, CDS900-R026, CDS1300-R032.

3.2 Register the non-collection tag namespaces

Adopt the CDS-SHOPIFY-0.2 tag namespace profile:

Prefix	Purpose	Generation
<code>collection_</code>	Automated collection / navigation projection	Computed from category rules
<code>merch_</code>	Merchandising signals (<code>merch_new_arrival</code> , <code>merch_sale</code> , <code>merch_clearance</code> , <code>merch_featured</code> , <code>merch_unisex</code>)	Governed manual or rule-based
<code>season_</code>	Seasonal relevance where a binary signal suffices (<code>season_winter</code> , <code>season_summer</code>)	Controlled value / rule-based
<code>workflow_ / supplier_</code>	Internal signals	PIM-only by default; published by exception only

Anything with a *value* (size, colour, material, fit, room...) is **not a tag** — it is an attribute (Phase 2) filtered via metafields (Phase 6). If you catch yourself designing multiple tags for values of one thing, stop: that is an attribute. Keep the legacy Sale/Clearance discipline: `merch_sale` = temporary promotion on a continuing product; `merch_clearance` = end-of-line sell-through; never one collection for both. **Why:** structured values in metafields, binary signals in tags — the split that keeps Phase 6 filters clean. Implements: CDS900-R028, CDS900-R029, CDS900-R027 (owner + lifecycle per merch collection).

3.3 OWNER DECISION — tag suffix format (one-way door)

Choose one format for tag suffixes and record it in the organisation profile: **human-readable** (`collection_Shirts`, mixed case, spaces allowed — readable in Shopify admin) or **slug**

(collection_shirts, lowercase-hyphen — friendlier to Liquid and programmatic use). CDS vo.2 does not mandate either; what it mandates is that every published tag is generated, registered or explicitly approved. **The choice is permanent in practice** — changing it later means bulk-updating every product, collection rule and filter config. Standing recommendation: human-readable, matching the legacy store's proven convention. **Why:** collection rules in Phase 5 must match tag strings exactly, forever. Implements: CDS900-R029; formatting is an organisation-profile declaration (CDS900-R005).

3.4 Record the tag registry and its ownership contract

Write the full tag registry (every tag, namespace, source: computed vs manual, purpose) into the organisation profile / PIM. Declare tag-list ownership now: **PARTITIONED_PIM** — the PIM owns the collection_*, merch_*, season_* partitions and regenerates them on every publication; tags written by other apps (reviews, loyalty, shipping) are externally owned and must survive publication. This matters because Shopify's productUpdate tag write is a **full replacement**: the connector must read the current downstream tag list, retain externally owned members, regenerate the PIM-owned partitions, and write the merged set — or use an explicitly additive operation — and must declare which in the mapping. **Why:** a partial tag write on a full-replace API silently deletes every other tag on the product. Implements: CDS900-R029, CDS900-R030, CDS500-R025, CDS500-R026, CDS500-R027; platform fact per CDS-900 §4.6 [S7].

Phase 4 — Shopify Store Bootstrap

Now — with taxonomy, dictionaries and tag design settled — touch Shopify.

4.1 Base store configuration

Set store details, markets, currency (AUD), taxes, shipping, checkout, legal pages, and the theme skeleton. Keep the theme choice compatible with: metafield display on product pages, breadcrumbs, and Search & Discovery filter rendering (check before committing — some themes need apps or Liquid work for breadcrumbs). **Why:** mechanical groundwork; the theme constrains Phases 6–7. Implements: (platform housekeeping; no CDS requirement — retained from legacy runbook.)

4.2 Staff accounts and permissions — use the one real lever

Create staff accounts using the **verified permission granularity**: Shopify product permissions are View, View cost, Create and edit, Edit cost, Edit price, Export, Delete — **there is no field-level or per-product edit restriction on any plan**, and anyone with product edit rights can also add, edit and delete metafield definitions and values. Therefore:

1. **Withhold "Create and edit" products** from every staff member who does not operationally need it (fulfilment, support, finance). This is the only preventive lever that exists.
2. For staff who must have product edit rights, protection is **detection-only**: training plus the Phase 10 drift loop.
3. Train every product-editing staff member on exactly **which admin edits will be overwritten or flagged as drift** (all PIM-owned fields — in practice: everything descriptive).

- Record the resulting authority-cutover evidence per field class in the organisation profile: permission withholding = evidence class (a); drift detection = class (b); accepted detection-only with named residual-risk owner = class (c). Shopify standard fields are always at best (b)/(c) — they cannot be write-protected. **Why:** honest protection on Shopify is one blunt permission plus detection; pretending otherwise is non-conformant. Implements: CDS900-R011, CDS1300-R014, CDS1300-R051; platform facts per CDS-900 §4.1 (verified 2026-08-04).

4.3 Install the PIM connector app with least privilege

Create the custom app (Admin API, GraphQL) for the PIM connector. Grant only the scopes the declared operations need (products, product listings, publications, metafields/metaobjects read-write; read-only elsewhere). Store credentials in secret management — never in product records, logs or exported evidence. Distinct permissions for destructive operations where supported. **Why:** the connector is the single authorised writer; its blast radius should match its contract. Implements: CDS500-R089, CDS500-R090, CDS500-R091, CDS500-R047.

4.4 OWNER DECISION — metafield edit-protection posture

For PIM-owned metafields there are two postures, either conformant if declared with its residual risk:

Posture	Protection	Cost
App-owned reserved namespaces	Shopify blocks all staff edits — the only edit-proof mechanism	Fields are invisible in Shopify admin : staff cannot see the data they are selling with; operator visibility loss is itself an error source
Merchant-visible metafields	None beyond permissions — any product-editor can edit or delete definitions and values	Full admin visibility of human-readable MF_* data; protection is training + drift detection after the fact

Standing recommendation (owner's stated preference): merchant-visible + drift detection.

It matches how the legacy pipeline actually worked — detection via Match fields, not prevention — and Phase 10's attribution workflow exists precisely to make detection fast and honest. Declare the chosen posture and its residual risk in the organisation profile / channel profile. **Why:** this is the D17 trade-off; the standard permits either but demands the declaration. Implements: CDS-900 §4.1 (D17 trade-off), CDS500-R077b context, CDS1300-R014(b).

4.5 Create custom metafield definitions

Create Shopify **custom metafield definitions** for every attribute that (a) will publish to Shopify and (b) has no suitable category metafield — per the schema-level choice recorded on each Attribute Definition in Phase 2 (the category-vs-custom choice is a schema decision, never per-product). Use a durable namespace/key (e.g. `cds.material_facet`, `cds.colour_facet`, `cds.room`, `cds.fit`) — the internal key may differ from the visible MF_ identifier. Every definition intended as a storefront filter must have a **filterable data type** (single-line text, boolean, numeric, metaobject reference). Expected set for this store: colour facet, material facet, fit, room, style, occasion, pattern, finish, shape, dimensions (numeric), care instructions (non-filter). Category metafields need no creation — they unlock automatically when a product is assigned its Standard Taxonomy category (Phase 9); prefer them where they exist and record

the choice per attribute. **Why:** filter groups in Phase 6 can only be built from proper metafield definitions. Implements: CDS900-R018, CDS900-R019, CDS900-R020, CDS900-R021; CDS-300 §8 (identifier vs machine key).

Phase 5 — Automated Collections

5.1 Create one automated collection per browsable taxonomy node

For every node customers will browse to, create an **automated** collection whose single condition is `product tag is equal to collection_<Node>` — the tag string matching the Phase 3 registry exactly, the collection name matching the tag minus the prefix. Set sort order deliberately (Best selling is a fine default). Manual collections are reserved for genuinely curated editorial sets.

Collection: Shirts	Condition: product tag equals collection_Shirts
Collection: Side Tables	Condition: product tag equals collection_Side Tables

Why: collections generated from the taxonomy maintain themselves; hand-curated taxonomy collections rot. Implements: CDS900-R026, CDS600-R013.

5.2 Create merchandising collections with owners and lifecycles

Create the overlay collections from `merch_*/season_*` tags: New Arrivals (`merch_new_arrival`), Sale (`merch_sale`), Clearance (`merch_clearance`), Featured (`merch_featured`), Winter Edit (`season_winter`), etc. For each, record in the organisation profile: **owner, membership rule (computed/curated/hybrid), activation window, retirement rule**. Keep Sale and Clearance separate collections, always. **Why:** merchandising collections without declared owners and lifecycles become permanent junk drawers. Implements: CDS900-R027, CDS600-R014, CDS600-R015.

5.3 Anti-pattern review

Before proceeding, check: no manual collection standing in for a taxonomy node; no two collections splitting one customer intent ("Hoodies" and "Sweatshirts"); no collection named differently from its tag rule; no per-brand collections (brand is a Vendor filter in Phase 6); no hand-maintained Sale collection. Shopify does not display filters on collections containing more than 5,000 products; split a collection before that limit is reached. **Why:** each anti-pattern is a future rebuild. Implements: legacy discipline under CDS900-R026–R028; limit-preflight spirit of CDS900-R035.

Phase 6 — Search & Discovery Filters

6.1 Install Search & Discovery and set the filter policy

Install Shopify's free **Search & Discovery** app. Policy, from the facet architecture: customer filters come from the **facet layer** — governed facet dictionaries projected into metafields — plus the useful built-ins. Configure **per collection context**, not one global set: apparel collections get Size / Colour / Fit /

Material / Price / Availability / Brand(Vendor); homewares collections get Colour / Material / Room / Style / Price / Availability, with dimensions where the category warrants (rugs, tables). **Never expose as filters:** raw tags of any namespace (`collection_*`, `merch_*`, `season_*` — S&D renders tags as one undifferentiated wall of internal strings), uncontrolled distinct values, or any attribute failing the eligibility test (decision-relevant? covered? discriminating? comprehensible? stable?). **Why:** filters are a curated projection, not an exposure of the data model. Implements: CDS900-R031, CDS900-R032, CDS600-R021, CDS600-R022, CDS600-R027, CDS600-R028, CDS200-R007.

6.2 Configure the colour filter as facet families

The colour filter uses the **facet family** metafield (Blue, Green, ..., per the Phase 2.2 declared set) — never the shade names. The product page continues to show the display shade ("French Navy"); the filter shows "Blue". Where S&D value grouping is used as a convenience, the **authoritative grouping remains the PIM facet dictionary** — S&D grouping is a channel-level mirror of it, never the master. **Why:** ~12 recognisable families filter; 80 supplier shade names do not. Implements: CDS900-R033, CDS900-R034, CDS600-R034, CDS600-R035, CDS1500-R017.

6.3 Preflight against the verified S&D limits

Record in the channel capability declaration and check every facet projection against the verified limits: **25 filters per store; 100 values displayed per storefront filter; 1,000 values visible per filter in the app; 200 unique values per filter group; 1,000 filter groups store-wide; no filters on collections over 5,000 products or searches over 100,000 results; the Category filter cannot use value grouping.** With the Phase 2 dictionaries none of these should bind — the preflight exists to catch future dictionary growth before Shopify rejects, hides or truncates a filter. **Why:** projections must be preflighted against declared platform limits, not discovered in production. Implements: CDS900-R035; limits per CDS-900 §4.7 [S2] (verified 2026-08-17).

6.4 Record facet definitions and their behaviour declarations

For every live filter, complete its Facet Definition in the PIM: stable ID, label, source attribute, eligible contexts, value dictionary, selection mode, **count unit and method** (including the disjunctive-count rule: an unselected value's count predicts the effect of adding it), ordering (sizes sort semantically by sort key, never alphabetically), disclosure default, owner. Declare at facet-set level: **cross-facet AND, within-facet OR** (the CDS defaults) and the response-time target. Declare the zero-result policy: values producing no truthful result are **disabled or hidden**; out-of-stock values may remain visible only with a clear unavailable state; zero-result pages preserve filters and offer removal of one constraint. Shopify note: native S&D evaluates variant-option and availability filters **at variant scope** (different filters AND at variant level), so the combination-availability capability is declared **present** in the channel profile. Per-value **count accuracy** under combined selections is *not yet verified* — it is Empirical Check 1 in Phase 10. **Why:** undeclared filter behaviour becomes platform-default behaviour nobody chose. Implements: CDS600-R024, CDS600-R025, CDS600-R026, CDS600-R047, CDS600-R048, CDS600-R056, CDS600-R058–R062, CDS600-R041, CDS600-R064, CDS600-R115; capability declaration per CDS-900 §4.7 (resolved note) and CDS600-R054/R055.

6.5 Accessibility and mobile pass on the filter UI

Verify in the theme: filter groups and controls carry programmatic labels; selected/disabled states and counts are exposed to assistive technology; swatches always have text labels (colour is never the sole signal); keyboard focus order is logical and visible; mobile filter panel shows active-filter count, previews the result count on deferred apply, and preserves selections on close. **Why:** these are normative requirements, not polish. Implements: CDS600-R036, CDS600-R068, CDS600-R069, CDS600-R070, CDS600-R072, CDS600-R092–R098.

Phase 7 — Navigation Menu

7.1 Build the menu from the taxonomy, max three visible levels

Main menu mirrors the taxonomy: Level 1 = top nav items, Level 2 = dropdown, Level 3 = mega-menu columns. Navigation is a **curated entry**, not a full rendering — deep or thin nodes stay reachable via category pages and search without appearing in the menu. Labels are customer language, non-overlapping within a level; no "Other/General/Misc". **Why:** the menu is the primary place flat Shopify collections read as a hierarchy. Implements: CDS600-R009, CDS600-R010, CDS600-R011, CDS600-R017, CDS600-R018.

7.2 Every menu item links to a collection

No menu links to search queries, filtered URLs or static pages where a collection exists. This is what keeps navigation stable, merchandisable and SEO-viable. **Why:** collections are the durable navigation destinations; URLs and filters are not. Implements: CDS600-R013 (deterministic membership behind each destination); legacy rule carried forward.

7.3 Breadcrumbs, cross-links, parent/sibling links

Configure in the theme: breadcrumbs on every collection and product page showing one canonical path per product consistent with the primary category (Home > Women > Tops > Shirts > Relaxed Linen Shirt); sub-collection tiles above the grid on parent pages; "Back to [parent]" and "Also in [parent]" sibling links on child pages; "Shop more [collection]" on product pages. Merchandising overlap (the unisex hoodie) is served by cross-links and `merch_*` collections — never a second primary path. **Why:** breadcrumbs and cross-links are how a flat collection model communicates structure and rescues direct-landing visitors. Implements: CDS600-R088, CDS600-R089, CDS600-R090, CDS600-R016, CDS200-R022.

7.4 Mobile three-tap test and faceted-SEO policy

Test on a phone: any Level 3 collection reachable in ≤ 3 taps; back-navigation at every level; < 8 items per menu level. Then declare the faceted-URL policy in the organisation profile: which facet combinations are indexable (default: none — canonical collection URLs only; facet selections do not mint crawlable URLs), and confirm the category/collection structure is crawlable independently of any facet URLs. **Why:** unmanaged facet URLs create an unbounded duplicate crawl space. Implements: CDS600-R104–R107; mobile discipline per CDS600-R068–R072.

Phase 8 — PIM Channel Wiring

8.1 Create the Shopify channel profile in the PIM

Record the channel profile: write mode (GraphQL Admin API), read mode (API read-back), operation semantics per object class, rate limits, **propagation window per write mode** (bounds PENDING and drives Amber→Red escalation), identity fields (Shopify product/variant/metafield IDs — retained after creation), **critical field set** (recommended: title, price, availability, primary image, `collection_*` tag partition), and the highest assurance level claimed (target: PA-7). **Why:** every downstream behaviour in Phase 10 keys off these declarations. Implements: CDS500-R011, CDS500-R012, CDS500-R013, CDS900-R036.

8.2 Define field mappings under the common mapping contract

For every published field, record the ten contract properties: canonical source, target path, transformation, requiredness, cardinality, **write semantics**, read-back path, comparison rule, ownership, failure policy. Shopify-specific write semantics that must be encoded, not assumed:

- **productUpdate does not update variants.** Variant data goes through `productVariantsBulkUpdate` or `productSet`. The connector must use the correct current mutation per object class.
- **Tags are full-replace** on `productUpdate`; the connector writes the merged set per the Phase 3.4 partition contract (or uses `tagsAdd` additively) and declares which.
- Tags and list fields compare as **sets**; rich text under a declared canonicalisation; references by stable IDs, not display text.
- Each MF_ attribute's projection (category metafield / custom metafield / native field) is the schema decision recorded in Phase 2/4 — the canonical MF_ prefix never changes because of the target.
Why: connectors written against assumed platform behaviour silently stop updating variants and silently delete tags. Implements: CDS900-R008, CDS900-R010, CDS900-R012, CDS900-R013, CDS900-R030, CDS900-R038, CDS900-R039, CDS900-R040, CDS500-R025; platform facts per CDS-900 §4.1 [S6], §4.6 [S7].

8.3 Variant and option projection rules

Only SKU-producing differences become Shopify variant options (size, sellable colour). The connector preflights projected option count (≤ 3) and variant count ($\leq 2,048$ documented; themes/channels may support less) against the declared limits **before** publication; anything exceeding them uses a documented decomposition strategy, never silent truncation. Do not create variants merely to make something filterable — that is the facet layer's job. **Why:** the variant boundary is a canonical-model rule; Shopify limits are a preflight, not a surprise. Implements: CDS900-R022, CDS900-R023, CDS900-R024, CDS200-R008; limits per CDS-900 §4.5 [S4].

8.4 Google Merchant Center wiring

Configure the GMC projection (direct feed/API from the PIM, or via Shopify's channel with the reduced observation coverage honestly declared):

- **color = the landing-page colour value** — the Display Label / sellable colour ("French Navy"), **never the facet family**. Submitting "Blue" for a French Navy landing page risks disapproval.
 - When colour identifies a variant, also publish Google's `variant_option` alongside `color`.
 - The broad searchable colour name goes in **title** where discoverability is wanted.
 - **Multi-colour:** up to 3 slash-separated values, one primary + up to two secondary ("Navy/White") — never commas or merged strings.
 - `CH_google_product_category` from the Phase 1.6 category mappings (ID or breadcrumb, one form); `CH_google_product_type` derived from the internal category path.
 - `item_group_id` groups variants consistently across updates; GTINs published wherever they exist (omitting an existing GTIN limits visibility); brand required for new products.
 - GMC disapprovals/diagnostics are imported as **observed channel quality events** — never written into canonical fields; a feed accepted into GMC is not "verified" — unobserved fields are UNOBSERVABLE with a coverage ratio. **Why:** GMC compliance rules are dated platform facts; the facet family is a storefront-filter concern only. Implements: CDS900-R042–R050; colour rule per CDS-900 §5.2 resolved note [G4] (verified 2026-08-04).
-

Phase 9 — Products: Pilot Cohort, Then Waves

9.1 Enter the pilot cohort in the PIM

Pick a representative pilot: one apparel family with size+colour variants (e.g. women's shirts), one homewares family with dimensions (e.g. side tables), one multi-colour print product (cushion), one product with a taxonomy-decision-log entry. Enter them PIM-first: primary category from Phase 1; variant values as canonical `value_ids`; attributes per the Phase 2 definitions; editors manually add **only** `merch_*/season_*` tags — `collection_*` tags and all projections are computed. Requiredness preflight blocks publication of R-level gaps; unknown supplier values enter quarantine, never silently extend dictionaries. **Why:** a pilot proves the whole chain end to end before scale-out. Implements: CDS1300-R015, CDS200-R015, CDS200-R011, CDS1500-R010, CDS900-R058; wave pattern per CDS-1300 §16.

9.2 Publish the pilot and verify structural landing

Publish via the connector. For each pilot product confirm in Shopify admin and storefront:

1. Standard taxonomy category assigned → **category metafields appeared** on the product edit page.
2. Product appears in every expected automated collection (its full category path + any merch collections) and no others.
3. Variants landed with correct option values, prices, SKUs (via the variant-capable mutation).
4. Metafield values landed under the intended definitions.
5. Externally owned tags (if any test tags were set) survived the tag write. **Why:** structural verification before data verification; each check maps to a Phase 4–8 decision. Implements: CDS900-R013, CDS900-R015, CDS900-R026, CDS900-R030; unlock behaviour per CDS-900 §4.3 [S1].

9.3 Finalise S&D filters against real category metafields

Return to Search & Discovery: with pilot products carrying category metafields, add any category-metafield-based filter sources that were unavailable pre-products (S&D lists metafields in use), confirm each filter group shows governed facet values only, and re-run the 6.3 limit preflight. **Why:** some filter sources only become selectable once products exercise the definitions. Implements: CDS900-R018, CDS900-R021, CDS900-R035.

9.4 Expand by wave with entry/exit criteria

Scale out by category wave (tops → dresses → knitwear → ...; cushions → tableware → lighting → ...). Each wave declares scope, entry criteria, exit criteria, rollback point, evidence package, and a **proving period** (post-cutover observation window that must elapse defect-free — one full business cycle recommended) before exit. Quality denominators include failed and quarantined records — completeness over a silently reduced population is non-conformant. **Why:** bounded, evidenced waves are the adoption pattern; the pilot is wave zero. Implements: CDS1300-R039, CDS1300-R041, CDS1300-R046, CDS1300-R047, CDS1300-R055–R057.

Phase 10 — Switch On Publish → Observe → Verify

This phase turns the store from "synced" into "verified". The loop: canonical revision → expected projection → preflight → dispatch → acknowledgement → **independent observation** → comparison → drift classification → governed repair.

10.1 Arm scheduled read-back and comparison

Enable the observation schedule: read back every PIM-owned published field via the Admin API (never the outbound payload, never a write-populated cache), on a cadence where **critical-set fields verify more frequently** than descriptive fields, respecting the declared propagation window (in-window latency = PENDING, not drift). Store snapshots with capture time, source and channel identity — separately from canonical values, **append-only**. Comparisons run under each field's declared registry comparator (money round-2dp, NORMALIZED_TEXT, UNORDERED_SET for tags — substring containment is never equality). **Why:** acknowledgement is not proof; observation is the only honest basis for "it's live and correct". Implements: CDS500-R002, CDS500-R003, CDS500-R035, CDS500-R048, CDS500-R049, CDS500-R050, CDS500-R051, CDS500-R052, CDS500-R055, CDS500-R058, CDS500-R077a, CDS900-R037.

10.2 Wire statuses, traffic lights and escalation

Surface results per field using the CDS-500 status enum exactly (MATCH / MISSING / MISMATCH / PENDING / UNOBSERVABLE / NOT_APPLICABLE / OVERRIDDEN / ERROR) with reason codes, presented as traffic lights: GREEN = MATCH only; **AMBER** = **missing/pending/unobservable/overridden** (absence is Amber, never Red); RED = confirmed MISMATCH or ERROR. Configure escalation: Amber escalates to Red presentation when the propagation window lapses unresolved or the field is in the critical set; any critical-set Red caps the product/channel aggregate at Red. Every indicator exposes its detailed status and reason code — Amber is never a generic

bucket. **Why:** the single normative status model, applied without local invention. Implements: CDS900-R041, CDS500-R060–R073, CDS500-R084–R086.

10.3 Empirical Check 1 (queued vertical-slice task) — S&D count accuracy under combined selections

Variant-scope filtering is verified present; **per-value count accuracy under combined selections is not** — this store is the test bench. On a pilot collection, construct the classic trap: a shirt with Blue only in S/M and White only in M. 1. Select Colour: Blue + Size: M → the trap product must be **excluded** (no purchasable Blue/M variant). 2. With Blue selected, read the displayed count next to each unselected Size value → each count must equal the number of products that would actually remain after adding that size (disjunctive semantics), counted in the declared unit. 3. Repeat with Availability toggled and a third facet (Material) active. Record outcomes as evidence in the channel profile. If counts are inaccurate, update the profile's capability declaration honestly (counts approximate; combination filtering accurate) and reflect the declared count method in the Facet Definitions. **Why:** the declared count semantics must match observed platform behaviour, or the declaration must change. Implements: CDS600-R052, CDS600-R053, CDS600-R054, CDS600-R056, CDS600-R057; open item per CDS-900 §4.7 resolved note and CDS-600 §16 resolved note.

10.4 Empirical Check 2 (queued vertical-slice task) — drift-attribution correlation workflow

Prove the detection-plus-attribution loop end to end with a controlled staff edit:

1. Have a staff account edit a PIM-owned field in Shopify admin (e.g. change a metafield value and a product title).
2. Confirm the next observation cycle classifies both as drift type **EXTERNAL_EDIT** with the correct previous observed value, new observed value and detection interval (append-only history makes this recoverable).
3. Run the correlation: narrow the detection interval against the Shopify **Store activity log** (recent window, view-only) and any `products/update` webhook timestamps (`X-Shopify-Triggered-At` — payload has *no* actor field) and the Events query (product field edits may generate **no** event).
4. Record the attribution outcome class on the drift record: **actor identified / actor class identified / unattributable**. All three outcomes are valid; the record must be honest.
5. Confirm attribution never delayed detection, status or escalation, and that repair follows the governed policy (auto-republish is permitted only for **EXCLUSIVE_PIM**/partition-owned fields).
Why: on Shopify, attribution is best-effort correlation by design — this check proves the workflow and calibrates expectations before real drift arrives. Implements: CDS500-R076, CDS500-R077a–c, CDS500-R078, CDS500-R080, CDS500-R028; attribution mechanisms and limits per CDS-900 §4.1 (verified 2026-08-04).

10.5 Run the proving period and hand over to run-state

Operate the full loop through the declared proving period. Exit requires: verification health above approved thresholds; the two empirical checks recorded; run-state ownership accepted by named owners (same people is fine under the small-team profile — the acceptance record is not waived); temporary

credentials rotated or retired; runbooks for publishing, verification triage, dictionary review and taxonomy change in place. **Why:** migration ends when the operating model is owned, not when the last job completes. Implements: CDS1300-R041, CDS1300-R044, CDS1300-R045, CDS1300-R054, CDS1300-R056, CDS1300-R057.

Completion Checklist — Vertical-Slice Acceptance (handoff §13)

The store is done when every box below is checkable with evidence, not opinion. The first five are the vertical-slice acceptance criteria; the rest are this runbook's phase gates.

Vertical-slice acceptance criteria:

- [] **PIM authoritative.** Every governed product fact has exactly one declared authority; descriptive attributes, classification and mappings are PIM-owned; Shopify edits to PIM-owned fields are detected as drift, not absorbed. (*CDS200-R027, CDS500-R028, CDS900-R011*)
- [] **Reproducible expected state.** For any published product, the expected Shopify representation can be regenerated from the same canonical revision + mapping version + dictionary version and matches what was stored at dispatch time. (*CDS500-R015, CDS500-R022*)
- [] **Read-back stored separately.** Observed Shopify values live in observation records (OBS_ -scope), append-only, never overwriting canonical values; adoption of an observed value is only ever an explicit, attributed change. (*CDS500-R005, CDS500-R077a, CDS200-R031*)
- [] **Differences classified.** Every non-MATCH field carries a core status from the eight-value enum plus a reason code; drift carries a cause category; missing and unobservable are never conflated; traffic lights follow the CDS-500 mapping with Amber escalation armed. (*CDS500-R054, CDS500-R060–R065, CDS500-R070, CDS900-R041*)
- [] **Clean facet family on filters, display shade on the product page.** The storefront colour filter shows the governed facet families (Blue); the product page and card show the display shade ("French Navy"); the GMC feed carries the landing-page value; the three layers remain distinct, related values in the PIM. (*CDS900-R033, CDS600-R033–R035, CDS1500-R014, CDS-900 §5.2*)

Phase gates:

- [] Taxonomy documented; one primary category per product; tie-break order declared; decision log live and enforced. (*Phase 1*)
- [] Dictionaries seeded from the CDS Reference Dictionary package and governed; colour facet set declared; attribute definitions with requirement levels, source layers and comparators in place; size systems governed. (*Phase 2*)
- [] Tag registry recorded; suffix format declared; tag ownership PARTITIONED_PIM with merged full-replace writes proven (external tag survives publication). (*Phase 3, 9.2*)
- [] Staff permissions applied per the withhold-product-edit lever; metafield posture declared with residual risk; authority-cutover evidence class recorded per field family; staff trained on what counts as drift. (*Phase 4*)
- [] One automated collection per browsable node, rules matching the registry exactly; merch collections have owners and lifecycles. (*Phase 5*)

- [] S&D filters per collection context from facet metafields only; verified limits preflighted; facet definitions complete with count semantics and zero-result policy; accessibility pass done. *(Phase 6)*
- [] Menu ≤3 levels, every item a collection, mobile 3-tap pass, breadcrumbs canonical, faceted-SEO policy declared. *(Phase 7)*
- [] Channel profile with propagation window and critical set; mappings declare write semantics (variant mutation, tag merge); variant preflight armed; GMC colour/title/multi-colour rules encoded. *(Phase 8)*
- [] Pilot cohort published and structurally verified; waves defined with proving periods; quality denominators include quarantined records. *(Phase 9)*
- [] Publish→observe→verify running on schedule; Empirical Check 1 (S&D count accuracy) and Empirical Check 2 (drift-attribution correlation) executed and recorded; run-state accepted by named owners. *(Phase 10)*

Appendix A — One Attribute End-to-End: the Worked Example *(informative)*

The whole system, for one product fact, with the exact names you would type. The fact: **the colour of a cushion** — the most universal filter in retail, and the example every CDS document shares. Every dictionary-backed, filterable attribute follows this identical shape — swap the vocabulary and the names. (Simpler attributes — free text, numbers, measurements — skip the dictionary and facet parts but keep the same definition → value → publish → verify spine.)

Step 1 — The dictionary (already shipped)

In `CDS_Reference_Dictionary/colour_reference_values.csv`, the colour vocabulary (field key `colour`) holds, among its shades:

Code (what software stores)	Name (what people see)	Filter family	Also matches (aliases)
french_navy	French Navy	blue	french navy, dark navy
navy	Navy	blue	navy blue, dark blue

The filter family points into the second colour vocabulary, `colour_facets.csv` (field key `colour_facet`), where `blue` displays as **Blue** — the broad family customers filter by, so the filter never shows forty near-identical shades.

Step 2 — The PIM attribute definition (once, at Phase 2)

Create one Attribute Definition — the record that gives the field its behaviour (CDS-200 §7):

Property	Value	Notes
Identifier	<code>MF_colour</code>	Semantic identifier — CDS-300 registry, <code>MF_</code> = structured attribute
Human label	Colour	What operators and storefronts display

Property	Value	Notes
Scope	product	Single-colour cushion; colour is variant-scoped only when it creates sellable variants
Value type	enum_reference	Values must come from a dictionary
Dictionary binding	colour	The vocabulary from Step 1 — nothing else is accepted
Requiredness	R (Required) for homewares decor	CDS-1500 §4 requirement levels; publication blocked when absent
Facet policy	filterable via the colour family	Customers filter Blue , never the forty-shade list
Comparison strategy	exact match of each projected channel value, whitespace/case-normalised	Used by read-back verification (Step 5)

Channel source layers — declared per CDS200-R026: which projection each destination receives:

Destination	Receives	For this cushion
Shopify display metafield	display label	French Navy
Shopify filter metafield	facet family	Blue
Google Merchant color	display label	French Navy — the landing-page value; the family is never sent (CDS-900 §5.2)

Step 3 — The product record (every product, mostly via import)

On the cushion's PIM record: `MF_colour = french_navy`. That code is the canonical fact; every visible form of it is looked up, never typed.

Import auto-match: a supplier CSV says `Dark Navy` → the alias matches (case-insensitively) → the PIM stores `french_navy`, recording provenance (import + supplier + matched alias). A supplier writing an unknown name — the standard's own famous case is `Moonlit Harbour` — does **not** create a value: it lands in the quarantine queue for a human, who either aliases it to an existing shade or promotes it as a new one (CDS-400 §17). Because colour is Required here, the cushion does not publish until resolved.

Step 4 — The Shopify side (once, at Phase 4)

Create two **product metafield definitions** (the store's declared metafield posture applies — Phase 4.4):

	Definition 1 — Display colour	Definition 2 — Colour family (the filter)
Namespace and key	<code>custom.colour_display</code>	<code>custom.colour_family</code>
Type	single line text	single line text
Validation	restricted to the dictionary's display labels (French Navy, Navy, ...)	restricted to the family labels (Blue, Green, ... — the 18-family baseline)
Filtering	—	the Search & Discovery filter source, so the storefront filter offers Blue (CDS-900 §4: text metafields are filter sources)

Operational note: capabilities — custom product metafield definitions, choice validation, storefront access, S&D filtering on text metafields — are the CDS-900-verified facts; the admin clicks to create them are Shopify UI detail not recorded in the dated profile — follow Shopify's current metafields help.

Step 5 — Publish, read back, verify (Phase 8/10, automatic)

Field	Expected (projected from french_navy)	Observed (read-back)	Result
custom.colour_display	French Navy	French Navy	status MATCH → shown Green
custom.colour_family	Blue	Blue	status MATCH → shown Green
CH_google_color	French Navy	per Google's observability (diagnostics/coverage declared honestly)	verified to the declared coverage

If a staff member edits the display metafield to "Navy" (a valid choice — but wrong for this cushion), the next read-back shows expected ≠ observed → status **MISMATCH** (shown Red — the colour of the light is presentation over the status), reason code CDS_VALUE_DIFFERS, drift class EXTERNAL_EDIT — with the previous observed value and the detection window recorded (CDS500-R077a). Correlating that window against the Store activity log to attribute *who* made the edit is the separate follow-up step (CDS500-R077b; best-effort on Shopify per CDS-900).

And the end state is the acceptance sentence of the whole standard: the product page says **French Navy**, the filter says **Blue**, Google gets the landing-page value, and every one of those is verifiably derived from one code on one record.

Implements: CDS200-R023–R024, R026 (attribute definitions incl. channel source layers), CDS400 dictionary binding + §17 quarantine, CDS300 registry naming, CDS900-R012/R028 + §4 metafield guidance + §5.2 Google color, CDS500 §16–§18 verification, CDS500-R077a (observation history; attribution correlation per R077b).

This runbook is informative companion material to CDS v0.2. Platform behaviour statements inherit the verification dates of CDS-900 (references retrieved 2026-08-03/04); re-verify against the current dated profile before relying on them in a later release.